

Structural Vision AR: Intelligent Structural Health Assessment and Virtual Building Preview Platform

Kishore P

1VA23CS082

Department of CS&E

Sai Vidya Institute of

Technology

Bengaluru, India

kishorep.23cs@saividya.ac.in

Sanjana Aithal

1VA23CS143

Department of CS&E

Sai Vidya Institute of

Technology

Bengaluru, India

sanjanaaithal.23cs@saividya.ac.in

Kiran R

1VA23CS081

Department of CS&E

Sai Vidya Institute of

Technology

Bengaluru, India

kiranr.23cs@saividya.ac.in

Sujan B R

1VA23CS161

Department of CS&E

Sai Vidya Institute of

Technology

Bengaluru, India

sujanbr.23cs@saividya.ac.in

Prof. Deepika G

Assistant Professor

Department of CS&E

Sai Vidya Institute of

Technology

Bengaluru, India

deepika.g@saividya.ac.in

Dr. Manjunath T N

Professor

Department of CS&E

Sai Vidya Institute of

Technology

Bengaluru, India

manjunath.tn@saividya.ac.in

Abstract: Manual visual inspection remains the dominant method for assessing the structural health of civil infrastructure, yet it is slow, subjective, and dependent on scarce domain expertise. This paper presents Structural Vision AR, a mobile platform that combines a two-stage deep learning pipeline with marker-less augmented reality to deliver on-site structural assessment from a single photograph. A YOLOv8-based instance segmentation model detects and delineates surface cracks at pixel level, while a MobileNetV3-Small classifier fine-tuned by transfer learning identifies the affected structural component (beam, ceiling, column, slab, or wall). A rule-based risk engine fuses crack area ratio with detection confidence to grade severity as Low, Medium, or High and to map each grade to a recommended maintenance window. The results are anchored back onto the physical structure through an ARCore-based viewer that projects the segmented crack pattern and risk-colored markers onto detected surfaces. The complete system is served by a FastAPI backend with Supabase-managed authentication, storage, and per-building scan history. Experimental evaluation shows a mask mAP@50 of 0.741 with precision of 0.851 for crack segmentation and 97% validation accuracy for component classification, with end-to-end scan latency under 3 seconds on

CPU-only hardware. The platform demonstrates a practical, low-cost pathway for digitizing preliminary structural inspection.

Keywords: Structural health monitoring, crack detection, instance segmentation, YOLOv8, MobileNetV3, transfer learning, augmented reality, ARCore, FastAPI, ONNX.

I. INTRODUCTION

Civil infrastructure such as residential buildings, bridges, and industrial facilities deteriorates continuously under load, weathering, and material aging. Surface cracking is among the earliest and most informative indicators of structural distress, and its timely identification directly influences maintenance cost and occupant safety. In practice, however, crack assessment is still carried out through periodic manual inspection: an engineer visually examines the structure, judges severity from experience, and records findings by hand. This process is time-consuming, expensive, difficult to scale across large building stocks, and inherently subjective, with assessments varying between inspectors and inspection cycles.

Advances in deep learning have made automated crack detection increasingly viable. Convolutional neural networks (CNNs) have been shown to outperform classical image-processing techniques for

concrete crack identification, and modern single-pass detectors extend this capability to real-time segmentation of crack geometry. In parallel, augmented reality (AR) frameworks on commodity smartphones now support marker-less plane detection and world-anchored rendering, allowing analysis results to be visualized directly on the physical structure rather than on a detached report.

Most existing implementations, however, address only one side of this opportunity: they either detect cracks in isolation or employ AR purely for design visualization. Few systems combine pixel-level crack segmentation, structural component identification, severity grading, maintenance forecasting, and in-situ AR visualization within a single deployable mobile application.

This paper presents Structural Vision AR, an end-to-end platform that closes this gap. The main contributions are as follows:

- A two-stage inference pipeline pairing a YOLOv8 instance segmentation model for crack detection with a MobileNetV3-Small transfer-learning classifier for structural component identification, both deployable on CPU-only server hardware.
- A rule-based risk engine that converts segmentation output into a Low/Medium/High severity grade and an actionable maintenance window prediction.
- An ARCore-based visualization module that projects the actual detected crack pattern, rendered server-side as a transparent textured overlay, onto real surfaces along with risk-colored 3D markers.
- A complete system architecture — Flutter mobile client, FastAPI inference backend, and Supabase persistence layer — evaluated on real devices with documented accuracy, latency, and failure-mode analysis.

The remainder of this paper is organized as follows. Section II reviews related work. Section III details the system architecture and methodology. Section IV reports experimental results, and Section V concludes with directions for future work.

II. RELATED WORK

Automated visual inspection of civil structures has been studied extensively across computer vision,

structural engineering, and, more recently, mobile computing. Early approaches relied on hand-crafted image processing — edge detectors, thresholding, and morphological operations — which degrade quickly under varying illumination, surface texture, and stain-like artifacts. Deep learning has since become the dominant paradigm for crack detection [1, 2].

A. Deep Learning for Crack Detection

Cha et al. [1] demonstrated that a CNN-based sliding-window classifier detects concrete cracks with over 97% accuracy while remaining robust to lighting and shadow variation, and later extended the approach with region-based detection of multiple damage types using Faster R-CNN [2]. Zhang et al. [5] trained one of the first deep networks for road crack detection on smartphone imagery, showing that learned features transfer better than hand-crafted ones. Public datasets accelerated this line of work: SDNET2018 [3] provides over 56,000 annotated concrete images, while the CRACK500 benchmark of Yang et al. [4] supplies pixel-level crack masks and established feature-pyramid architectures for thin-structure segmentation. Instance segmentation frameworks such as Mask R-CNN [8] made pixel-accurate delineation practical, and one-stage successors in the YOLO family [6] now offer comparable mask quality at a fraction of the inference cost, which is decisive for CPU-bound deployments such as ours.

B. Component Recognition and Transfer Learning

Assessing severity requires knowing not only that a crack exists but where it exists: a crack in a load-bearing column carries different implications than one in a partition wall. Lightweight CNNs designed for mobile and edge inference, notably the MobileNet family [7], achieve strong classification accuracy with a few million parameters. Combined with transfer learning from large-scale pretraining [10], such networks can be fine-tuned to specialized domains — here, five structural component classes — with modest labeled data. A known limitation of deployed classifiers is overconfidence on out-of-distribution inputs [9], which we analyze explicitly in Section IV.

C. Augmented Reality in Construction and Inspection

AR adoption in the construction sector has grown around design overlay and progress monitoring, with industrial systems pairing BIM models with head-

mounted displays, and a research agenda for AR/VR in architecture, engineering, and construction has been articulated by Davila Delgado et al. [11]. Smartphone AR frameworks such as ARCore provide marker-less plane detection, hit-testing, and world anchoring without dedicated hardware. However, published systems rarely feed machine-learning inspection output into the AR view. Structural Vision AR differs by rendering the segmentation result itself — the detected crack polygons — as a world-anchored overlay, closing the loop between automated analysis and on-site visualization.

III. METHODOLOGY

The proposed system follows a client-server architecture in which a Flutter mobile application captures imagery, a FastAPI backend performs all machine-learning inference, and Supabase provides authentication, relational storage, and image hosting. Figure 1 shows the system architecture and Figure 2 the end-to-end processing workflow.

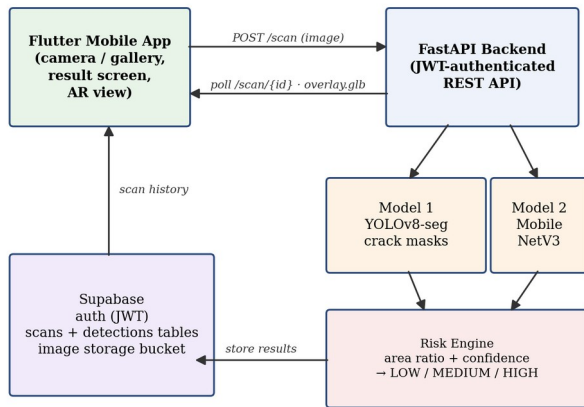


Figure 1: System Architecture of Proposed Model

3.1 System Architecture and Scan Lifecycle

A scan begins when the user captures or uploads a photograph of a structural element. The image is transmitted as a JWT-authenticated multipart request to the backend, which processes the scan asynchronously: the client polls the scan endpoint every two seconds until inference completes. Both models are loaded by file path at startup, so a retrained model can be swapped in by overwriting a single file with no code change — an operationally important property during iterative retraining. Results, including crack polygons, component label, confidence scores, and the computed risk grade, are persisted to Supabase per user and per building, enabling

longitudinal tracking of crack progression across repeated scans.

3.2 Dataset Preparation and Pre-processing

The crack segmentation model is trained on a merged corpus built from the CRACK500 pavement crack dataset [4] and a public Roboflow concrete crack segmentation dataset, unified into YOLO polygon-annotation format. Merging heterogeneous sources broadens the distribution of surfaces, lighting conditions, and crack widths seen in training. Images are letterboxed to the training resolution, and standard augmentations (scale jitter, horizontal flip, HSV perturbation, mosaic) are applied by the training framework. For the component classifier, a five-class image dataset (beam, ceiling, column, slab, wall) was assembled and split into training and validation sets; classifier inputs are resized to 256 pixels, center-cropped to 224×224, and normalized with ImageNet statistics to match the pretraining domain [10].

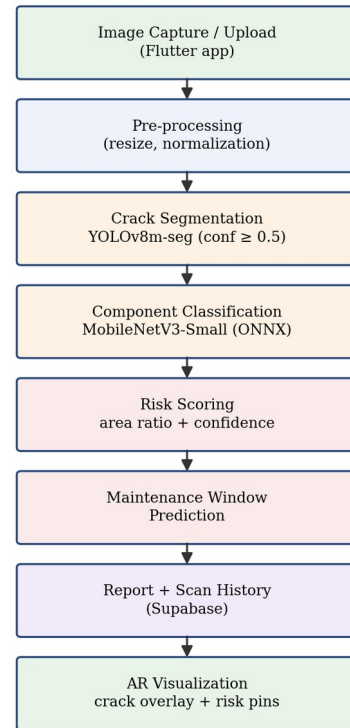


Figure 2: Workflow of Proposed Methodology

3.3 Model 1 — Crack Instance Segmentation

Crack detection is performed by a YOLOv8 segmentation network [6], which predicts bounding boxes, per-instance polygon masks, and confidence scores in a single forward pass. Two versions were developed iteratively. Version 1 used the small YOLOv8s-seg variant trained at 640 px on a single public dataset and reached a mask mAP@50 of 0.691.

Error analysis showed the model was data-bound rather than architecture-bound, with failures concentrated on thin hairline and low-contrast cracks under-represented in training data. Version 2 therefore attacked the data axis first and capacity second: the merged dataset of Section 3.2, a higher 1024 px training resolution so hairline cracks survive downscaling, and a one-step capacity increase to YOLOv8m-seg — large enough to absorb the additional data, yet small enough to preserve the sub-3-second CPU response budget. At inference the confidence threshold is set to 0.5, as the default 0.25 produced false positives on crack-like textures such as seams and shadow edges.

3.4 Model 2 — Structural Component Classification

The component classifier is a MobileNetV3-Small network [7] initialized with ImageNet weights and fine-tuned on the five-class component dataset, then exported to ONNX for fast CPU inference through ONNX Runtime. The network learns a feature hierarchy in which early layers respond to material textures, intermediate layers to shape and orientation cues that separate the classes — a column as a vertical elongated region against background, a beam as its horizontal counterpart, a ceiling by upward perspective — and a final softmax produces the class distribution. The maximum class probability is surfaced in the application as an interpretable confidence value.

3.5 Risk Scoring and Maintenance Window Prediction

Severity grading fuses two signals from the segmentation output: the crack area ratio (total detected crack polygon area divided by image area), which drives severity, and detection confidence, which escalates the grade only when the affected area is already substantial — high confidence on a tiny crack means “definitely a crack,” not “dangerous.” Table 2 lists the decision rules, tuned by validating the resulting grade distribution over a held-out set of crack images. Each grade maps to a maintenance window prediction — Immediate (0–6 months), Short-term (6–24 months), or Long-term (2–5 years) — translating raw detection output into an actionable inspection recommendation.

Risk Level	Condition and Maintenance Window
HIGH	Area ratio > 0.05, or > 0.03 with

Risk Level	Condition and Maintenance Window
	confidence > 0.85 — Immediate (0–6 months)
MEDIUM	Area ratio > 0.01, or > 0.005 with confidence > 0.80 — Short-term (6–24 months)
LOW	Otherwise — Long-term (2–5 years)

Table 2: Risk grading rules and maintenance windows

3.6 AR Visualization Module

The AR viewer is built on ARCore through a Flutter AR plugin. On entering AR mode the application performs marker-less plane detection, displaying tracked feature points immediately with guidance prompts to improve first-use success. Two visualization elements are anchored to detected surfaces. First, risk-colored 3D pin markers, generated server-side as GLB assets so marker styling can change without an application update, are placed by tap on any tracked plane. Second — and the module's distinguishing feature — the backend renders each scan's detected crack polygons onto a transparent texture, constructs a flat quad mesh on demand, and serves it as a GLB; the client projects this overlay onto the tapped surface so the user sees the actual segmented crack pattern registered on the physical wall, rather than a generic annotation. The platform additionally supports placement of a preset building model on an empty site for virtual building preview.

IV. RESULTS

The system was evaluated at deployment settings (640 px inference) on a fixed 200-image validation set for segmentation, on the held-out classification split for the component model, and end-to-end on a physical Android device (Vivo Y200) with the backend running on CPU-only hardware (Intel 13th-gen i7, no discrete GPU).

4.1 Crack Segmentation Performance

Table 1 compares the two model versions on the common validation set. The retrained YOLOv8m-seg model improves mask mAP@50 from 0.691 to 0.741, driven almost entirely by an 11-point precision gain to 0.851 — the deployed model now rarely hallucinates cracks. Recall held at 0.700, indicating that residual errors are false negatives concentrated in hairline and low-contrast cracks still under-represented in training data; this diagnosis directs the

improvement roadmap discussed in Section V. Inference cost roughly doubled to ~390 ms per image on CPU, comfortably within the < 3 s end-to-end scan budget, and the previous model is retained as an instant rollback.

Metric	v1 (YOLOv8 s-seg)	v2 (YOLOv8 m-seg)	Change
Mask mAP@50	0.691	0.741	+0.050
Mask precision	0.741	0.851	+0.110
Mask recall	0.700	0.700	±0.000
CPU inference (ms)	~190	~390	+200

Table 1: Crack segmentation results across model versions

4.2 Component Classification and Failure Analysis

The MobileNetV3-Small classifier achieved 97% validation accuracy across the five component classes. Occlusion-based attention analysis confirms the model attends to semantically meaningful regions: for a correctly classified column (confidence 0.97), the attention mass falls on the vertical member itself, as shown in Figure 3.



Figure 3: Correct column classification (left) and occlusion attention map (right)

A documented failure case illustrates the classifier's principal limitation. A cluttered bedroom wall photographed with furniture and partial occlusion was classified as ceiling with 0.98 confidence (Figure 4): the occlusion heatmap shows the model relied on a plain, uniformly lit upper surface region resembling a ceiling, while contextual objects contributed nothing toward the correct class. Such overconfidence on out-of-distribution input is a well-documented property of deep classifiers [9] rather than a defect specific to this model. Mitigations adopted are in-app framing guidance — photograph the element so it fills the frame, as an inspector naturally would — and, longer term, augmenting training data with cluttered real-world scenes.

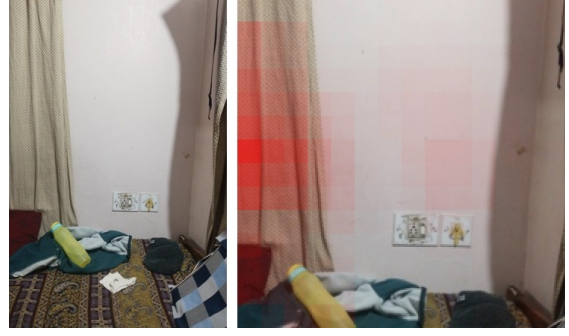


Figure 4: Out-of-distribution failure — cluttered wall classified as ceiling (0.98), with attention concentrated on the ceiling-like plain region

4.3 Risk Grading Output

Figure 5 shows representative pipeline outputs across the three risk levels drawn from the curated evaluation set: a heavily cracked surface with area ratio 0.112 graded HIGH, a single moderate crack at 0.025 graded MEDIUM, and a hairline crack at 0.008 graded LOW. Verified against the live pipeline, the grades matched the expected label for all six curated images (two per level). The higher small-crack sensitivity of the v2 model shifts area ratios upward, skewing the wider validation distribution toward MEDIUM — an interaction between detector recall and fixed thresholds that motivates threshold re-tuning after each retrain.

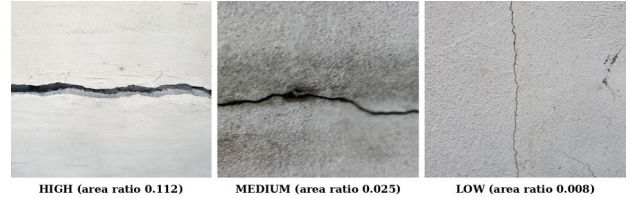


Figure 5: Representative scans across the three risk levels

4.4 On-Device AR and System Performance

On the test device, the release build detects the first plane in approximately one minute of scanning under normal indoor lighting (an order of magnitude faster than debug builds), after which pin placement and crack-overlay projection are interactive. The complete scan round-trip — upload, dual-model inference, risk grading, and result retrieval — completes in under 3 seconds. Device testing also drove robustness fixes, including camera handover between the scanning and AR subsystems and a point-hit fallback when a tap misses tracked plane geometry. Current limitations are that plane detection is restricted to horizontal surfaces on the test device pending an ARCore update, and the crack overlay is placed at the user's tap point at a fixed physical size

rather than being automatically registered to the true crack position.

V. CONCLUSION AND FUTURE WORK

This paper presented Structural Vision AR, a mobile platform that unifies pixel-level crack segmentation, structural component classification, rule-based risk grading, maintenance window prediction, and in-situ augmented reality visualization in a single deployable system. The two-stage pipeline achieves a mask mAP@50 of 0.741 with 0.851 precision for crack segmentation and 97% accuracy for component classification, with end-to-end latency under 3 seconds on CPU-only hardware — demonstrating that meaningful structural inspection support does not require specialized equipment. The documented model evolution shows that the current accuracy ceiling is set by training data rather than architecture, and the transparent failure-mode analysis of out-of-distribution inputs reflects a deliberately honest deployment posture.

Future work proceeds along four axes. First, closing the recall gap toward a 90% mAP target through targeted data collection on observed failure patterns, auditing noisy public-dataset labels, deploying at the native 1024 px training resolution, and tuning the detection operating point. Second, enriching the AR module with per-crack markers and severity labels, vertical plane support, true spatial registration of overlays to physical cracks, and physical crack-width measurement from AR plane geometry. Third, distinguishing cosmetic surface crazing from structurally significant cracking using geometric features of the segmented masks — width, straightness, and branching — augmented by AR-measured physical width. Fourth, a video scan mode that samples frames from a short sweep and merges detections, extending single-image assessment to whole-room coverage. Together these directions advance the platform from a per-photograph assessment tool toward a comprehensive digital inspection assistant.

REFERENCES

- [1]. Y.-J. Cha, W. Choi, and O. Büyüköztürk, "Deep Learning-Based Crack Damage Detection Using Convolutional Neural Networks," *Computer-Aided Civil and Infrastructure Engineering*, vol. 32, no. 5, pp. 361–378, 2017, doi: 10.1111/mice.12263.
- [2]. Y.-J. Cha, W. Choi, G. Suh, S. Mahmoudkhani, and O. Büyüköztürk, "Autonomous Structural Visual Inspection Using Region-Based Deep Learning for Detecting Multiple Damage Types," *Computer-Aided Civil and Infrastructure Engineering*, vol. 33, no. 9, pp. 731–747, 2018, doi: 10.1111/mice.12334.
- [3]. S. Dorafshan, R. J. Thomas, and M. Maguire, "SDNET2018: An annotated image dataset for non-contact concrete crack detection using deep convolutional neural networks," *Data in Brief*, vol. 21, pp. 1664–1668, 2018, doi: 10.1016/j.dib.2018.06.015.
- [4]. F. Yang, L. Zhang, S. Yu, D. Prokhorov, X. Mei, and H. Ling, "Feature Pyramid and Hierarchical Boosting Network for Pavement Crack Detection," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 4, pp. 1525–1535, 2020, doi: 10.1109/TITS.2019.2910595.
- [5]. L. Zhang, F. Yang, Y. D. Zhang, and Y. J. Zhu, "Road crack detection using deep convolutional neural network," 2016 IEEE International Conference on Image Processing (ICIP), Phoenix, AZ, USA, 2016, pp. 3708–3712, doi: 10.1109/ICIP.2016.7533052.
- [6]. G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," Ultralytics, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
- [7]. A. Howard et al., "Searching for MobileNetV3," 2019 IEEE/CVF International Conference on Computer Vision (ICCV), Seoul, Korea, 2019, pp. 1314–1324, doi: 10.1109/ICCV.2019.00140.
- [8]. K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," 2017 IEEE International Conference on Computer Vision (ICCV), Venice, Italy, 2017, pp. 2980–2988, doi: 10.1109/ICCV.2017.322.
- [9]. A. Nguyen, J. Yosinski, and J. Clune, "Deep neural networks are easily fooled: High confidence predictions for unrecognizable images," 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Boston, MA, USA, 2015, pp. 427–436, doi: 10.1109/CVPR.2015.7298640.
- [10]. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," 2009 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Miami, FL, USA, 2009, pp. 248–255, doi: 10.1109/CVPR.2009.5206848.
- [11]. J. M. Davila Delgado, L. Oyedele, P. Demian, and T. Beach, "A research agenda for augmented and virtual reality in architecture, engineering and construction," *Advanced Engineering Informatics*, vol. 45, 2020, 101122, doi: 10.1016/j.aei.2020.101122.